

Authored by: Arpit Raj, LNMIIT Jaipur | **Date:** July 29, 2026

© Copyright Arpit Raj. All rights reserved.

Number of Provinces (LC_547)

Asked in: Samsung, Navi, Intuit

Notes highly inspired by my language and code

Algorithm (DFS)

- Create a seen array.
- Traverse every city.
- If Aadz reaches a city that hasn't been visited:
 - A new province is found.
 - Increment answer.
 - DFS from that city.
 - DFS marks the entire connected province as visited.
- Continue checking remaining cities.

Algorithm (BFS)

- Create a seen array.
- Traverse every city.
- If the city is unseen:
 - Increment provinces.
 - Push it into the queue.
 - Perform BFS.
 - BFS visits every city belonging to that province.
- Continue until all cities are visited.

Complexity

DFS

Time: $O(n^2)$

Space: $O(n)$ (visited + recursion stack)

BFS

Time: $O(n^2)$

Space: $O(n)$ (visited + queue)

Code Implementation (BFS)

```
class Solution {
public:

    int findCircleNum(vector<vector<int>>& isConnected) {

        int n = isConnected.size();
        int ans = 0;

        vector<int> seen(n + 1, 0);
        for(int currcity = 0; currcity < n; currcity++){

            if(!seen[currcity]){ //pehli baar node dekha , new province

                ans++;
                queue<int> q; //ab isse connected jitne bhi cities he unko
mark kr lenge
                seen[currcity] = 1;
                q.push(currcity);

                while(!q.empty()){
                    int curr = q.front();
                    q.pop();

                    for(int newcity = 0; newcity < n; newcity++){
                        if(isConnected[curr][newcity] == 1){
                            if(!seen[newcity]){
                                seen[newcity] = 1;
                                q.push(newcity);
                            }
                        }
                    }
                }
            }
        }

        return ans;
    }
};
```

Code Implementation (DFS)

```
class Solution {
public:

    int n;
    void dfs(vector<vector<int>>& isConnected, vector<int>& seen, int city){

        seen[city] = 1;
        for(int newcity = 0; newcity < n; newcity++){

            if(seen[newcity]) continue;
            if(isConnected[city][newcity] == 1){
                dfs(isConnected, seen, newcity);
            }
        }
    }

    int findCircleNum(vector<vector<int>>& isConnected) {

        n = isConnected.size();
        vector<int> seen(n, 0);
        int ans = 0;

        for(int currcity = 0; currcity < n; currcity++){
            if(!seen[currcity]){
                ans++;
                dfs(isConnected, seen, currcity);
                //to mark all the connected components as seen
                //because wo bhi same province ka part he
            }
        }

        return ans;
    }
};
```